

Weekly Report Compilation

Taehee Kim, SNU DLLab, Prof. Tae-Wan Kim

Summary of work over seven weeks. The first four weeks cover the TrustCore hardware root of trust, from algorithm study through RTL, verification, synthesis and FPGA bring up. The last three weeks cover a shift toward robotics perception, learning ROS 2 and studying lidar based SLAM with a view to accelerating part of the pipeline on FPGA.

Week 1: TrustCore concept evaluation and C reference models

- Reviewed the TrustCore concept as a hardware root of trust and scoped which functions the chip had to provide
- Studied SHA-256 and Ascon at the algorithm level: message schedule and compression rounds for SHA-256, and the permutation, associated data handling and tag generation for Ascon
- Implemented both algorithms in C and checked the output against published test vectors
- Kept the C versions as reference models so that RTL output could later be compared byte by byte

Result: a working software reference, plus a clearer view of which parts of each algorithm map cleanly to hardware and which are expensive.

Weeks 2 and 3: Verilog implementation, SystemVerilog verification, Genus synthesis

- Implemented the cores in Verilog along with the surrounding chip logic: an SPI slave front door, buffer control, and a command FSM that routes traffic to either the SHA-256 or the Ascon core
- Defined the SPI packet format (command, length, key, nonce, data) and the result read back path
- Wrote the testbench in SystemVerilog, driving SPI transactions and comparing results against the C reference vectors
- Covered normal operation as well as edge cases such as zero length input, back to back commands and unrecognised command codes
- Ran synthesis in Cadence Genus for the target CMOS process and clock, and reviewed gate count, area and timing reports

Result: functional RTL matching the software reference, and area numbers showing the design did not fit the die budget the team originally assumed, which fed directly into the decision to change direction.

Week 4: Vivado and FPGA bring up on ZCU106

- Ported the design to Vivado targeting the ZCU106 board
- Built a block design with the Zynq UltraScale+ processing system driving the crypto logic through an AXI interface, and packaged the design as a custom IP block
- Added board constraints so that results are visible on the board itself, with LEDs for pass and fail and pushbuttons to trigger each core
- Ran synthesis and implementation, worked through build and constraint errors, and reviewed resource utilisation and timing

Result: a working path for checking processor to logic behaviour on real hardware instead of relying on simulation alone.

Week 5: ROS 2 fundamentals

- Went through the basics: nodes, topics, publisher and subscriber patterns, launch files, and workspace builds with colcon
- Practised recording and replaying data with rosbag, which is the main way to work with sensor data without a live setup
- Looked at the standard message types and driver structure for lidar and IMU data

Result: able to run, inspect and modify existing ROS 2 pipelines, which is the prerequisite for the SLAM work in the following weeks.

Weeks 6 and 7: SLAM study and data collection support

- Read papers on SLAM and lidar inertial odometry, mainly LIO-SAM, along with background material on scan matching and factor graph optimisation
- Traced where computation time actually goes in these pipelines, from point cloud preprocessing and feature extraction through to scan matching and back end optimisation
- Considered how the ZCU106 fits: the processing system running ROS 2, with a single well chosen kernel offloaded to the programmable logic rather than the whole pipeline
- Helped Kim Jayhyeon with sensor data collection so that there are real recorded sequences to work from

Result: a narrower and more realistic idea of what to accelerate, and recorded data to evaluate it against.

Current direction

Following on from the SLAM study, the plan is to narrow the scope to a single stage rather than a full odometry system: a real time noise filter on lidar range data in programmable logic, compared against a CPU implementation on the same board.